Closure Conversion

 © 2025 Александра Лановая.
(редактировал: Косарев Дмитрий)

Creative Commons Attribution 4.0 International

1. Введение и терминология

Во многих языках программирования функции не являются *полноценными first-class citizens*: возможности оперировать ими ограничены. Например, в C можно передавать указатель на функцию в качестве аргумента и возвращать функцию как результат. Однако функции в C не могут быть вложенными и должны определяться только на верхнем уровне. Это упрощает их реализацию — по сути, функция в C представляется просто указателем на код. В функциональных языках (Scala, OCaml, Haskell, Scheme и др.) функции могут быть вложенными и способны переживать область видимости, в которой были определены. Это предоставляет широкие возможности (например, функция может возвращать новые функции), но и усложняет реализацию: простых указателей на код уже недостаточно.

Чтобы проиллюстрировать проблему, рассмотрим такой пример на псевдокоде:

```
def f(x):
    def g():
        return x
    return g

a = f(10)
b = f(20)
print(a())  # должно напечатать 10
print(b())  # должно напечатать 20
```

Здесь внутренняя функция `g` использует переменную `x`, определённую во внешней функции `f`. При двух вызовах `f` мы получаем два разных экземпляра функции `g`: `a` замыкает `x = 10`, а `b` — `x = 20`. В силу лексической области видимости вызов `a()` должен вернуть 10, а `b()` — 20. Чтобы обеспечить это, необходим специальный механизм: простой перенос `x` в глобальную переменную не подходит (иначе оба замыкания вернули бы 20). Этим механизмом и является **замыкание**.

Замыкание (closure) состоит из двух частей: *кода функции* и *окружения* — набора переменных со значениями, которые эта функция захватывает из внешних контекстов. Создание замыкания можно понимать так: берётся открытое λ-выражение (функция, использующая свободные переменные) и снабжается окружением со значениями этих переменных, превращаясь в самодостаточный функциональный объект. Обычно замыкание реализуют как структуру с двумя полями: указатель на код (функцию) и указатель на структуру окружения со значениями всех свободных переменных [1]. Благодаря этому функция может использовать внешние значения даже после выхода из исходной области видимости, соблюдая принципы лексической (статической) области видимости [2].

Свободные и связанные переменные. Переменная называется *свободной* в функции (λ-выражении) f, если она используется в теле f и при этом не является ни её параметром, ни локально определённой внутри неё. Формально, если $f = \lambda(p_1, \dots, p_n)$. e и L — множество переменных, объявленных внутри e, то

$$FV(f) = \{\, v \mid v \text{ встречается в } e,\ v \notin \{p_1, \dots, p_n\} \cup L \,\}.$$

Свободные переменные берутся из внешнего контекста (окружения). Например, в примере выше для внутренней функции `g` переменная `x` является свободной, а для `f` все переменные внутри неё — связанные (параметр `x` в `f` связан). Противоположное понятию свободной — *связанная переменная*, то есть параметр или локальная переменная, объявленная внутри функции. Все свободные переменные функции образуют тот самый набор, который сохраняется в её окружении при преобразовании программы.

Closure Conversion. *Closure Conversion* (CC, преобразование замыканий) — этап компиляции, на котором программа переписывается так, чтобы все функции стали *замкнутыми* (то есть не содержали свободных переменных). После CC каждая функция явно получает все необходимые для своего выполнения данные — либо через дополнительные параметры, либо через поля окружения. CC выполняется до финальной генерации кода, обычно на

промежуточном представлении программы (например, CPS или ANF). Часто closure conversion сочетают с *lambda lifting*: после того как все функции стали замкнутыми, их можно вынести из вложенных контекстов на верхний уровень (как глобальные функции). Это упрощает дальнейшую генерацию кода в языках без вложенных функций (например, на C). Важно не путать эти два шага: сначала выполняется closure conversion, и только затем, опционально, — lambda lifting [3].

Два подхода к реализации замыканий. Существуют две основные стратегии, различающиеся организацией окружения и алгоритмом преобразования [2, 1]:

- *«Плоское» замыкание (flat closure)* — хранит все захваченные значения непосредственно в объекте замыкания (например, в кортеже или массиве слотов) вместе с указателем на код. Окружение интегрировано в сам объект замыкания и не ссылается на другие структуры.

- *Замыкание с явным окружением (explicit environment)* — хранит окружение отдельно, как выделенный в куче объект (record) с полями под каждую захваченную переменную. В объекте замыкания хранится только указатель на такой объект-окружение. Если функция вложена, её окружение может ссылаться на окружение внешней функции (образуя цепочку) вместо дублирования общих данных.

Далее рассмотрены общий алгоритм closure conversion, а затем подробно описаны два указанных варианта: плоский подход (алгоритм top-down) и подход с явным окружением (алгоритм bottom-up).

2. Общая схема преобразования (Closure Conversion)

На этапе closure conversion компилятор переписывает программу так, чтобы **удалить все свободные переменные**, введя вместо них явные окружения. Основные шаги:

1. **Анализ свободных переменных.** Вычисляется множество $\mathrm{FV}(\mathrm{f})$ для каждой функции (λ-выражения) в программе. Это делается обходом AST: для каждой λ собираются все переменные, которые используются в её теле и не объявлены в ней (ни как параметры, ни локально). Если функция содержит вложенные λ, их свободные переменные учитываются: например, если внутренняя функция использует переменную ν

из внешней, то v входит в FV внутренней функции (а значит, v придётся сохранить в окружении).

2. **Формирование окружений.** Для каждой функции, у которой $\mathrm{FV} \neq \emptyset$, определяется представление окружения. Создаётся структура (запись или массив) с полями для каждой переменной из FV. Если FV пусто (функция не имеет свободных переменных), окружение ей не требуется — такую функцию можно представить просто указателем на код.

3. **Переписывание λ-выражений и вызовов.** Каждое λ-выражение вида $(\lambda\ (\texttt{params}).\ \texttt{body})$ с ненулевым FV заменяется на выражение, которое *создаёт замыкание* — пару {код; окружение}. Кодом становится новая функция, куда свободные переменные добавлены как дополнительные параметры (чаще всего первый параметр — `env`, неявный параметр окружения). Окружение формируется как объект, содержащий текущие значения всех свободных переменных. Например:

$$\lambda x.\, x{+}a{+}b \quad\longrightarrow\quad \texttt{make_closure}\Big(\lambda(\texttt{env}, x).\, x{+}\texttt{env.a}{+}\texttt{env.b},\ \texttt{make_env}(a,\ b)\Big),$$

   где `make_env(a,b)` создаёт объект окружения с полями `a = a` и `b = b`, а в теле функции используются `env.a` и `env.b` вместо исходных `a` и `b`.

   Все вызовы функций тоже переписываются: вместо прямого вызова `f(10)` выполняется вызов через раскрытое замыкание. Например, вызов `f(x)` преобразуется так, чтобы передать функции её окружение:

$$\texttt{tmp} = \mathrm{f};\quad \texttt{tmp.code}(\texttt{tmp.env},\ 10).$$

   Здесь мы распаковали замыкание `f` на `code` и `env`, а затем вызвали `code`, передав ему дополнительный аргумент `env`.

4. **Корректность преобразования.** После переписывания программа должна сохранять исходную семантику. Замыкания гарантируют, что каждая копия функции получает собственное окружение с нужными значениями, поэтому лексическая (статическая) область видимости не нарушается. Инвариант корректности CC: каждое обращение к переменной в преобразованной программе ссылается либо на локальное определение, либо на поле окружения, соответствующее исходной свободной переменной, и это поле содержит правильное (лексически ожидаемое) значение.

3. Flat Closure Conversion (Top-Down)

Идея плоского замыкания: код + слоты

Подход **flat closure** представляет каждое замыкание как единый *плоский* объект в памяти: в нём хранятся **указатель на код функции** и **значения всех её свободных переменных**. Реализация такого замыкания обычно подразумевает размещение структуры в непрерывном блоке памяти (например, массиве), где первый элемент — адрес кода, а последующие элементы — значения захваченных переменных. Такой объект эквивалентен кортежу: первый компонент — функция, а остальные — связанные с ней данные.

При вызове функции, реализованной как flat-замыкание, обращение к захваченным переменным выполняется напрямую через разыменование этого объекта. Например, если замыкание `cl` хранит `k`-ое захваченное значение, то `cl[k]` вернёт его. Таким образом, доступ осуществляется за один шаг (без дополнительных указателей). Цена такого упрощения — больший расход памяти: каждое замыкание получает собственные копии значений, даже если одни и те же данные нужны нескольким замыканиям.



Рис. 1: Структура замыкания (flat)

Алгоритм Top-Down (спуск сверху вниз)

Алгоритм closure conversion *top-down* обрабатывает программу, спускаясь от внешних функций к вложенным. Для каждой функции (по мере обхода дерева сверху вниз):

- Вычисляется множество её свободных переменных. Анализ FV можно выполнить заранее, либо проводить во время обхода.
- Если $FV \neq \emptyset$, **сразу создаётся структура окружения** для этой функции. В контексте flat-реализации это означает выделение блока памяти

нужного размера и заполнение его текущими значениями всех переменных из FV. В переписанном коде вместо исходного λ-выражения вставляется выражение, которое создаёт замыкание.

$$\lambda\texttt{params.body} \longrightarrow \{\texttt{code} = \lambda(\texttt{env}, \texttt{params}).\,\texttt{body}', \quad \texttt{env} = [\text{значения FV}]\},$$

где body' — это body с заменёнными упоминаниями переменных из FV на поля объекта env.

- Тело функции (body') преобразуется рекурсивно. Поскольку мы движемся от внешнего контекста к внутреннему, к моменту обработки вложенной функции её внешняя функция уже преобразована и имеет собственное окружение. Если внутренняя функция захватывает переменную внешней, то при создании её замыкания будет использовано текущее значение этой переменной (копия из окружения внешней функции). Таким образом, каждое вложенное замыкание получает свою копию всех необходимых данных.

Пример: Flat CC (Top-Down)

Рассмотрим функцию высшего порядка, возвращающую две вложенные функции, и выполним для неё преобразование по схеме flat.

```
(* Функция mkPair возвращает пару функций,
каждая из которых использует захваченную переменную x *)
let mkPair = fun x ->
let add = fun y -> x + y in (* первая вложенная функция
 ↪  *)
let mul = fun z -> x * z in (* вторая вложенная функция
 ↪  *)
(add, mul)
in
let (increment, double) = mkPair 1 in
Printf.printf "%d, %d\n" (increment 5) (double 5); (*
 ↪  ожидается: 6, 5 *)
```

В mkPair обе вложенные функции add и mul захватывают одну переменную x. После вызова mkPair 1 получаем пару замыканий increment и double, в каждом из которых должно сохраниться значение x = 1

(чтобы `increment 5` вернул 6, а `double 5` вернул 5). Выполним **closure conversion** по схеме flat. Для наглядности введём типы данных для замыканий:

```
(* Структура Closure: содержит код функции и указатель
 ↪  на окружение *)
type 'a closure = { code: 'a; env: env }
and env = { free_x: int }
```

Здесь `'a closure` — параметризованный тип (тип функции передаётся как параметр), а `env` — тип окружения с одним полем `free_x` для сохранения значения `x`. После преобразования функция `mkPair` переписывается так:

```
let mkPair = fun x ->
(* Создаем окружение для первой функции *)
let env_add = { free_x = x } in
let add_code = fun env y -> env.free_x + y in
(* Создаем отдельное окружение для второй функции *)
let env_mul = { free_x = x } in
let mul_code = fun env z -> env.free_x * z in
(* Возвращаем два замыкания с разными окружениями *)
({ code = add_code; env = env_add },
{ code = mul_code; env = env_mul })
in
let (increment, double) = mkPair 1 in
(* Вызов функций increment и double с передачей
 ↪  окружений *)
let res1 = increment.code increment.env 5 in (* вызовет
 ↪  add_code с env_add *)
let res2 = double.code double.env 5 in (* вызовет
 ↪  mul_code с env_mul *)
Printf.printf "%d, %d\n" res1 res2; (* выведет: 6, 5 *)
```

Обратите внимание: при flat-подходе значение `x = 1` было **скопировано дважды** — в поле `free_x` окружения `env_add` и в поле `free_x` окружения `env_mul`. Каждое замыкание `increment` и `double` хранит свою собственную копию `x`. Если бы таких функций было больше или они создавались в цикле, это привело бы к дублированию данных и дополнительным расходам памяти.

4. Explicit-Env Closure Conversion (Bottom-Up)

Идея явного окружения: объект в куче с полями

Альтернативный подход хранит окружение отдельно от объекта замыкания. Само замыкание представляет собой пару {код функции; указатель на окружение}. Окружение — самостоятельный объект (в куче) с именованными полями под каждое сохранённое значение. Например, на C можно определить:

```
struct Closure {
    FunctionPtr code;
    struct Env* env;
};
```

а окружение описать как структуру `Env` с полями для нужных переменных. При создании замыкания выделяется объект `Env`, и указатель на него сохраняется в структуре `Closure`.

Главное преимущество **explicit environment** — возможность **разделить один объект окружения между несколькими замыканиями**. Если во внешней функции определено несколько вложенных функций, захватывающих одни и те же переменные, компилятор может создать один общий объект окружения и поместить туда эти переменные. Все вложенные функции будут хранить указатель на этот общий объект вместо того, чтобы каждая хранила свою копию. Более того, вложенная функция может хранить указатель на окружение внешней функции, если ей нужны именно её переменные. В результате окружения могут образовывать **цепочку**: вложенное окружение ссылается на окружение внешней функции и т.д., благодаря чему не происходит дублирования данных.

Недостаток explicit-env подхода — дополнительная косвенность при доступе к данным. Функции приходится сначала перейти по указателю `env`, а затем взять нужное поле. Если окружения связаны цепочкой, для доступа к переменной внешнего уровня может потребоваться несколько разыменований. Таким образом, *явное окружение экономит память*, но потенциально увеличивает время доступа к захваченным переменным.

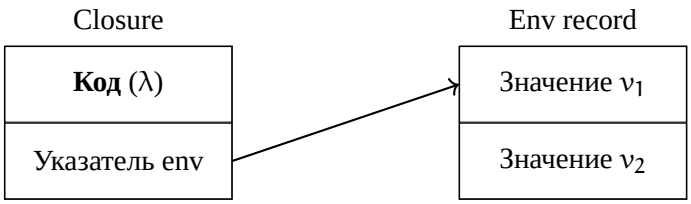


Рис. 2: Структура замыкания (explicit-env)

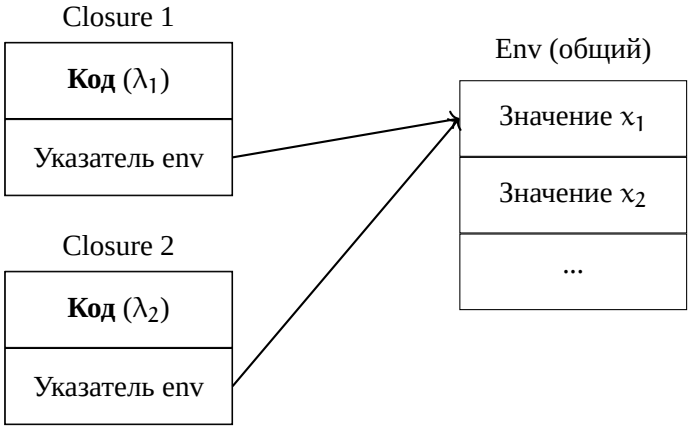


Рис. 3: Два замыкания, разделяющие одно окружение (общие данные)

Алгоритм Bottom-Up (от внутренних функций к внешним)

Второй вариант алгоритма CC работает *снизу вверх*, начиная с самых вложенных функций и поднимаясь к внешним. Его логика такова:

- Сначала преобразуются все **внутренние** (листовые) λ-выражения. Для них строятся замыкания с соответствующими окружениями.
- Возвращаясь на уровень внешней функции, анализируется, какие её переменные были захвачены внутренними функциями. Если вложенная функция использует локальную переменную x из внешней, то значение x должно сохраниться после завершения внешней функции (ведь замыкание может выполняться позже). Поэтому такие x добавляются к множеству свободных переменных внешней функции, даже если формально x не являлась для неё свободной. Таким образом, внешняя функция рассматривается как использующая дополнительные данные, необходимые её внутренним функциям.

- Для внешней функции формируется **объект окружения**, включающий:

  1. её собственные свободные переменные (переменные, которые она сама берёт из ещё более внешнего контекста);
  2. переменные, необходимые её внутренним функциям (выявленные на предыдущем шаге).

  Если внешняя функция сама является вложенным замыканием и получает какую-то переменную через своё окружение, то вместо копирования этого значения во внутренней функции можно просто передать ссылку на окружение внешней.

- После определения окружения внешняя функция переписывается: ей добавляется параметр окружения, и все обращения в её теле к переменным из FV (включая те, что нужны только вложенным функциям) заменяются на `env.field` или `env->field`. К этому моменту её тело уже содержит преобразованные вложенные функции (которые создают замыкания, указывающие на окружения).

- Алгоритм повторяется, поднимаясь к следующим уровням, пока не будет обработана вся программа.

Bottom-up стратегия требует большего анализа (учёта связей между функциями разных уровней), но минимизирует дублирование. Общие для нескольких замыканий значения сохраняются во внешнем окружении один раз, и несколько функций разделяют один объект окружения. В случае группы взаимно рекурсивных функций можно создать единое окружение для всей группы, избежав дублирования и циклических ссылок.

Пример: Explicit-Env CC (Bottom-Up)

Возьмём ту же функцию `mkPair` из предыдущего примера, где внутри определены две вложенные функции `add` и `mul`, использующие переменную `x`. После выполнения `mkPair 1` ожидается, что `increment` будет прибавлять 1, а `double` умножать на 1. После **closure conversion** с явными окружениями код `mkPair` будет выглядеть так (псевдокод):

```
let mkPair = fun x ->
(* Создаем один общий объект окружения *)
```

```
let env_shared = { x_val = x } in
(* Код вложенных функций; им добавлен параметр env *)
let add_code = fun env y -> env.x_val + y in
let mul_code = fun env z -> env.x_val * z in
(* Создаем замыкания, оба хранят указатель на env_shared
 ↪  *)
let add_closure = (add_code, env_shared) in
let mul_closure = (mul_code, env_shared) in
(add_closure, mul_closure)
in
let (inc, dbl) = mkPair 1 in
(* inc и dbl - это пары (code_ptr, env_ptr). Вызовем их
 ↪  корректно: *)
let res1 = (fst inc) (snd inc) 5 in (* вызовет add_code
 ↪  с env_shared и 5 *)
let res2 = (fst dbl) (snd dbl) 5 in (* вызовет mul_code
 ↪  с env_shared и 5 *)
Printf.printf "%d, %d\n" res1 res2; (* печатает: 6, 5 *)
```

Обе функции `inc` и `dbl` ссылаются на один и тот же объект окружения `env_shared`, внутри которого `x_val = 1`. Таким образом, `x` **не дублируется** — он хранится в памяти единожды и разделяется между этими замыканиями. Для сравнения, при flat-реализации этого примера каждая функция получила бы свою копию `x` (два отдельных окружения с `x = 1`). Подход с явным окружением позволяет сэкономить память и избежать дублирования данных.

Обращение к переменной `x` в теле функций `inc`/`dbl` осуществляется через цепочку указателей: сначала берётся `env_shared`, затем из него поле `x_val`. Если бы уровень вложенности был больше, могла бы потребоваться более длинная цепочка (например, `env_inner->env_outer->x`), что несколько увеличивает накладные расходы по времени.

5. Альтернативные подходы

В дополнение к рассмотренным flat closure и explicit environment существуют ещё два важных метода реализации замыканий, применяемых в специфических сценариях.

Первый метод — **дисплей** (display) или **chain links**. При входе в каждую функцию в глобальном массиве указателей по её уровню вложенности сохраняется адрес текущего окружения (активационной записи). Доступ к свободной переменной реализуется за $O(1)$: компилятор вычисляет индекс уровня вложенности, достаёт указатель из дисплея и по известному смещению читает значение переменной. При выходе из функции дисплей восстанавливается, возвращая указатели предыдущего уровня. Дисплей устраняет heap-окружения и множественные переходы по static link, но требует обновления массива при каждом вызове и возврате, что накладывает накладные расходы на выполнение [4].

Второй метод — **явная передача параметров окружения**. Вместо создания структур замыкания компилятор добавляет в сигнатуру вложенной функции все переменные, которые она захватывает из внешнего контекста, и при каждом вызове передаёт их как обычные аргументы. Рассмотрим пример преобразования функции `mkPair`, где внутренняя функция `add` захватывает переменную `x`:

```
let mkPair = fun x ->
  (* передаём x как явный параметр *)
  let add = fun x_env y -> x_env + y in
  (add x, add)
in
let (increment, add_func) = mkPair 1 in
let result = increment 5 in
Printf.printf "%d\n" result;  (* выведет 6 *)
```

Здесь `add` приобретает сигнатуру `fun x_env y → x_env + y`, где `x_env` — явно передаваемый параметр окружения. При вызове `mkPair 1` мы получаем `increment` как частично применённую функцию `add`, без каких-либо структур замыкания. Этот метод устраняет накладные расходы на выделение и управление окружениями в рантайме, но эффективен только при небольшом числе свободных переменных (при большем числе сигнатуры функций становятся слишком длинными, а функции нельзя безопасно возвращать за пределы контекста определения) [3].

6. Заключение

В заключение, рассмотренные методы преобразования замыканий демонстрируют различные компромиссы между временем доступа, расходом памяти и сложностью реализации рантайма. На практике современные компиляторы комбинируют эти подходы с учётом целей и характеристик целевого языка. В JavaScript-движках (V8, SpiderMonkey) применяется модель, близкая к flat closure: для каждой функции создаётся объект `Scope`, содержащий её переменные и цепочку лексических контекстов [5]. В компиляторах ML и Scala используют explicit environment с оптимизациями (environment trimming [6], escape analysis [7]) [8]. В высокопроизводительных реализациях при необходимости добавляют дисплей для прямого доступа к глубоким уровням вложенности [4].

Список литературы

[1] Mattew Might. *Closure conversion: How to compile lambda*. URL: https://matt.might.net/articles/closure-conversion.

[2] EPFL. *Closure conversion — Advanced Compiler Construction (CS-420)*. 2020. URL: https://cs420.epfl.ch/archive/20/c/05_cc.html.

[3] Thomas Johnsson. «Lambda lifting: Transforming programs to recursive equations». В: *Functional Programming Languages and Computer Architecture*. Под ред. Jean-Pierre Jouannaud. Berlin, Heidelberg: Springer Berlin Heidelberg, 1985, с. 190—203. ISBN: 978-3-540-39677-2.

[4] Alfred V. Aho и др. *Compilers: Principles, Techniques, and Tools (2nd Edition)*. USA: Addison-Wesley Longman Publishing Co., Inc., 2006. ISBN: 0321486811.

[5] V. Egorov. *Grokking V8 closures for fun (and profit?)* 2012. URL: https://mrale.ph/blog/2012/09/23/grokking-v8-closures-for-fun.html.

[6] Zhong Shao и Andrew W. Appel. «Efficient and safe-for-space closure conversion». В: т. 22. 1. New York, NY, USA: Association for Computing Machinery, янв. 2000, с. 129–161. DOI: 10.1145/345099.345125. URL: https://doi.org/10.1145/345099.345125.

[7] YM Tang. *Control-Flow Effects for Escape Analysis*. 1992. URL: https://www.cri.ensmp.fr/classement/doc/A-225.pdf.

[8] J. Gajarský. *Tutorial on Closure Conversion and Lambda Lifting*. 2014. URL: https://gist.github.com/jozefg/652f1d7407b7f0266ae9.